AI / GPU Cost Audit Checklist

22 leak points we see repeatedly in mid-market AI/ML cloud accounts

AICloudStrategist · Anushka B, Founder

April 2026

AI / GPU Cost Audit Checklist

22 leak points we see repeatedly in the AI/ML infrastructure of mid-market companies in 2026. Covers training, inference, managed API spend, feature stores, and governance.

> Built by AICloudStrategist · Founder-led. Enterprise-reviewed. · support@aicloudstrategist.com

---

How to use this checklist

If your monthly AI infrastructure bill is over ₹3L/month and nobody has run a structured review in the last six months, you almost certainly have 30–50% avoidable spend hiding in the line items below.

Work through the items. Mark each **DONE**, **NOT APPLICABLE**, or **ACTION REQUIRED**. Items with estimated saving above ₹50,000/month become Jira tickets before this document is filed.

The items are ordered by typical INR impact across the AI-first startups and AI-adopter mid-market companies we have reviewed.

Section 1 — Training (Items 1–6)

1. Fine-tuning jobs on On-Demand GPUs

What leaks: p4d.24xlarge at $32.77/hr On-Demand × 8-hour fine-tuning job × 10 jobs/week = ₹21L/month. Spot capacity in the same region is typically 70–85% available at a 60–70% discount. **How to detect:** SageMaker Training Jobs console → filter Instance Type starts_with “p” or “g5” or “g6” → sum of billable seconds × instance hourly rate. **Fix:** Migrate 70–85% of fine-tuning workloads to Spot with checkpointing. Test interruption recovery. Use Managed Spot Training (SageMaker) for automatic handling. **Typical saving:** ₹5L–₹15L/month at this spend band.

2. GPU utilisation below 40%

What leaks: A p4d.24xlarge running a training job that uses 6 of 8 GPUs at 40% utilisation = 1.5 effective GPUs for the price of 8. **How to detect:** NVIDIA DCGM + CloudWatch GPU metrics → Average GPU Utilization during training, per instance. **Fix:** Right-size to the smallest instance that fits

the model. For 7B models, g5.2xlarge beats p4d.24xlarge for fine-tuning by 6–8×. Use gradient accumulation to fit larger batch sizes on smaller instances. **Typical saving:** 50–75% on training compute.

3. Idle GPU instances

What leaks: Training instances left running after a job completes. p4d.24xlarge at $32/hr × 48 idle hours = ▫1.3L wasted per instance per weekend. **How to detect:**

```
aws cloudwatch get-metric-statistics --namespace AWS/EC2 \
  --metric-name GPUUtilization --statistics Average \
  --dimensions Name=InstanceId,Value=i-xxxx \
  --start-time <48hrs ago> --end-time now --period 3600
```

Fix: Auto-terminate training instances after 1 hour of GPU utilisation below 5%. SageMaker lifecycle hooks or custom Lambda. **Typical saving:** ▫50K–▫3L/month depending on fleet size.

4. Dataset in S3 Standard instead of Intelligent Tiering

What leaks: 10 TB of training data in S3 Standard ($230/month) accessed twice a quarter. Intelligent Tiering or Glacier Instant Retrieval is 80% cheaper. **How to detect:**

```
aws s3api list-objects-v2 --bucket <ml-data-bucket> \
  --query "Contents[?StorageClass=='STANDARD'].[Key,LastModified]" -
o text
```

Fix: S3 Lifecycle rule → transition objects untouched >30 days to S3 Intelligent Tiering. **Typical saving:** ▫30K–▫1.5L/month on datasets above 10 TB.

5. Checkpoints without retention policy

What leaks: Training runs saving checkpoints every 100 steps, accumulating in S3 indefinitely. A 70B model checkpoint is ~140 GB. 50 checkpoints = 7 TB. **How to detect:** S3 inventory report on model artefact bucket. **Fix:** Retention policy — keep the last checkpoint, keep checkpoints at epoch boundaries, purge the rest after 30 days. **Typical saving:** ▫20K–▫80K/month.

6. Redundant data preprocessing runs

What leaks: Data team reprocesses the same 500 GB dataset on every training run because the preprocessing pipeline isn't cached. **How to detect:** Audit ETL pipeline — is the output of preprocessing reused across runs, or regenerated? **Fix:** Cache preprocessed features in S3 with a content-hash key. Regenerate only when raw data changes. **Typical saving:** 20–40% of data-prep compute.

Section 2 — Inference (Items 7–13)

7. Real-time inference endpoints over-provisioned

What leaks: SageMaker endpoint on g5.2xlarge (▫78K/month) serving 3 req/s average, 12 req/s peak. g5.xlarge (▫36K/month) meets the SLA at peak. **How to detect:** SageMaker

endpoint metrics → Invocations per minute, Model Latency. SageMaker Inference Recommender can size automatically. **Fix:** Right-size endpoints. For variable traffic, enable autoscaling (min 1, max N based on peak capacity / concurrent request limit). **Typical saving:** 40–60% per endpoint.

8. No inference caching for repetitive prompts

What leaks: A summarisation endpoint sees 40% prompt repetition in a 24-hour window. Every repeated call costs full inference. **How to detect:** Log unique prompt hashes → measure duplicate rate over 7 days. **Fix:** Redis semantic cache (exact match) for deterministic prompts. For LLM endpoints, semantic similarity caching (using embedding similarity) captures rephrased duplicates. **Typical saving:** 20–50% on inference cost depending on prompt diversity.

9. Inference endpoints running 24×7 when traffic is 8×5

What leaks: Internal-tool inference endpoints running overnight and weekends with zero traffic. **How to detect:** Endpoint invocation metrics by hour-of-day. **Fix:** Serverless Inference (pay per request, cold start ~10s for small models) for sparse traffic. Or scheduled scale-down to 0 during known idle windows. **Typical saving:** 60–75% on non-24×7 endpoints.

10. Managed API spend without cost-per-feature tracking

What leaks: ₹8L/month on OpenAI / Anthropic / Bedrock, no visibility into which product features are driving which API calls. **How to detect:** Log API calls with a `feature` tag in the request metadata. Aggregate by tag monthly. **Fix:** Per-feature cost dashboard. Kill or throttle low-ROI features. Move high-volume features to self-hosted alternatives where cost/latency/quality math supports it. **Typical saving:** 15–40% by eliminating high-cost low-value features.

11. GPT-4 / Claude Opus for tasks that work on cheaper models

What leaks: GPT-4o at $5/1M tokens for a summarisation task that gives identical quality on Haiku at $0.25/1M tokens. Or Opus at $15/1M for extraction that works on Sonnet at $3/1M. **How to detect:** Sample 1000 production requests per feature → A/B test against a cheaper model → measure quality drop. **Fix:** Route by task complexity. Use cheaper models as the default, escalate to expensive models only when needed. **Typical saving:** 50–80% on managed API bill.

12. No streaming responses when latency matters

What leaks: User-facing LLM features using non-streaming responses — users wait, perceived latency is bad, abandonment increases. **How to detect:** Product telemetry on time-to-first-token. **Fix:** Enable streaming on all user-facing endpoints. Perceived latency drops from 8s to 400ms. **Indirect saving:** Higher engagement → better unit economics on a feature already in cost.

13. Multi-model endpoints not used for low-traffic variants

What leaks: Five low-traffic model variants on five separate endpoints (▫36K/month each) when SageMaker Multi-Model Endpoints would run them on a shared instance. **How to detect:** Endpoint inventory → flag endpoints with <10 req/min average. **Fix:** Consolidate onto Multi-Model Endpoints. Model load time adds ~200ms to first request after idle period. **Typical saving:** 60–80% on consolidated endpoints.

Section 3 — Feature Store and Data Pipeline (Items 14–17)

14. DynamoDB feature store in Provisioned mode at low utilisation

What leaks: 40,000 RCU / 40,000 WCU provisioned running at 11% utilisation. On-Demand billing would be 50–70% cheaper at that pattern. **How to detect:** CloudWatch → DynamoDB ConsumedReadCapacityUnits vs ProvisionedReadCapacityUnits. **Fix:** Switch to On-Demand if utilisation is under 40%. Or right-size Provisioned with Auto Scaling. **Typical saving:** 40–70% on feature-store spend.

15. Feature store serving stale features

What leaks: Real-time feature store recomputing features every minute when business logic requires hourly freshness. **How to detect:** Audit feature freshness requirements vs actual compute frequency. **Fix:** Match compute cadence to business need. Batch feature computation where daily is enough. Stream only the features that truly need <1-minute freshness. **Typical saving:** 30–60% on feature-compute cost.

16. Vector database oversized for query volume

What leaks: Pinecone pod-based plan ($72/month per pod × 8 pods) or managed vector DB with provisioned throughput for a workload seeing 50 queries/second. **How to detect:** Vector DB query volume metrics. **Fix:** Serverless vector DB tier. Or self-host (Qdrant / Weaviate / pgvector on standard Postgres) for workloads <1M vectors. **Typical saving:** 50–85% on vector DB cost.

17. SageMaker Processing Jobs oversized

What leaks: ml.m5.24xlarge processing jobs for preprocessing that fits on ml.m5.xlarge. **How to detect:** SageMaker Processing Jobs → CPU + memory peak utilisation. **Fix:** Right-size processing instances. Split large jobs across smaller instances (SageMaker Distributed Processing). **Typical saving:** 40–70% per job.

Section 4 — Governance (Items 18–22)

18. No cost-per-1K-inferences dashboard

What leaks: The team cannot answer “what does it cost to serve one user of Feature X for a month?” — so product pricing decisions are guesses. **How to detect:** Ask the question. If no dashboard exists, this is the action. **Fix:** Build a per-endpoint, per-feature cost dashboard on the cloud’s billing export. Tag every endpoint with `product_feature`. **Impact:** Unlocks all the above — and enables product-margin conversations.

19. No GPU approval workflow

What leaks: Data scientists provision p4d instances without platform team review. Shadow spend accumulates. **How to detect:** Check provisioning events in CloudTrail / Azure Activity Log / GCP Audit. **Fix:** Require platform team approval for any instance >g5.xlarge or >$10/hour. Enforce via IAM policy. **Impact:** Stops the leak at source.

20. No cost budget + alert per team or product

What leaks: Monthly AI spend is a single line item. One team’s experiment spikes, nobody notices until the monthly review. **How to detect:** AWS Budgets / Azure Budgets / GCP Budget Alerts — one per team tag. **Fix:** Per-tag budgets with alerts at 50%, 80%, 100% of monthly target. **Impact:** Early warning before waste compounds.

21. No build-vs-buy framework for managed AI APIs

What leaks: Every new feature is “let’s use OpenAI” or “let’s self-host” — decided on vibes, not numbers. **How to detect:** Ask the team for the spreadsheet that compares self-hosted cost vs managed API cost at projected 12-month volume. If it doesn’t exist, this is the gap. **Fix:** A reusable template: cost-per-request × volume, latency SLA, data privacy requirement, vendor lock-in risk, operational burden. We publish ours on request. **Impact:** Better decisions, documented reasoning.

22. Model versions without retention policy

What leaks: Every training run saves a model artefact. 200 model versions × 50 GB = 10 TB in S3. **How to detect:** S3 inventory on model registry bucket. **Fix:** Retention policy — keep production + staging + last 10 experimental versions. Archive the rest to Glacier. **Typical saving:** ₹25K–₹1L/month.

Summary

Section	Items	Typical Impact at ₹8L+/month AI spend
Training	1–6	30–50% of total savings
Inference	7–13	30–50%
Feature Store + Data Pipeline	14–17	10–20%
Governance	18–22	Unlocks the rest

Aggregate target: a typical Indian mid-market AI account working through this list closes 35–55% of monthly AI spend over 3–4 weeks of focused effort.

The three questions every AI leader should be able to answer

If you cannot answer these from your current dashboards, you have a visibility gap:

1. **What is your cost per 1,000 inferences, per endpoint?**
2. **What is your cost per training run, per model?**
3. **What is your cost per feature, per active user per month?**

These three numbers are the difference between “AI is expensive” (hand-wavy, untrue) and “this specific feature has a 23% margin at current pricing” (actionable).

If you want a second pair of eyes

Our **AI Architecture Review & Consulting** service (Service 08) covers all of the above in a structured 2–3 week engagement, with founder-led delivery and senior-architect oversight on every finding. Typical outcome at ₹15L–₹30L/month AI spend: ₹7L–₹12L/month reduction verified across three billing cycles.

Start with a free 30-minute **Cloud Cost Health Check** — same structure, lighter scope, written summary same day.

aicloudstrategist.com/book.html

AICloudStrategist · Anushka B, Founder · Rohini, Delhi 110085 · support@aicloudstrategist.com Founder-led. Enterprise-reviewed.

— Anushka B, Founder · AICloudStrategist